

Apigee X Simulator — Policy Test Guide, UI Walkthrough & Multi-Endpoint Testing

What's inside this document:

- A page-by-page walkthrough of the Apigee Simulator UI: Proxies, Environments, Target Servers, Products, Developers, Apps, KVM, References, Deployments, Trace, Analytics, and Monitoring.
- The core concepts that make this simulator behave like real Apigee X: **ProxyEndpoint**, **TargetEndpoint**, **Route Rules**, and **Conditional Flows** — with a real, working sample of each.
- A hands-on exercise: build a brand-new API proxy with two Target Endpoints, method-based Route Rules, and a Conditional Flow, entirely from the UI.
- A guide to testing multiple endpoints — route rules, conditional flows, and target-server failover — using both a pre-seeded demo proxy and the one you build yourself.
- A policy-by-policy testing reference covering all 24 policy types in the catalog: what each one does, exactly how to configure it in the UI, and a real sample request and response for each.

NOTE: Every sample in this document was generated from this project's actual source code (policyCatalog.ts, seed.ts, policyExecutors.ts) — not generic Apigee documentation. If your local install behaves differently from a sample here, that is a real discrepancy worth investigating.

1. Introduction

This guide is written so a colleague with no prior exposure to this project can sit down, run the application locally, and both use it and test it end-to-end. It assumes no familiarity with the Apigee Simulator's UI and walks through every screen before getting into policy testing.

The document is organized in the order you'll actually use it:

- 1 **Getting the app running** — the exact commands and login credentials.
 - 2 **The UI, screen by screen** — what every page does and how to use it.
 - 3 **Core concepts** — ProxyEndpoint, TargetEndpoint, Route Rules, and Conditional Flows explained with a real example.
 - 4 **Hands-on build exercise** — construct a new API proxy with two target endpoints, route rules, and a conditional flow, entirely from the UI.
 - 5 **Testing multiple endpoints** — how to prove route rules, conditional flows, target endpoints, and failover all work.
 - 6 **Policy-by-policy testing reference** — every policy type, how to configure it, and a sample request/response.
-

2. Getting the App Running

Run these once, from the project root, in two terminals (backend first):

Backend (Terminal 1)

```
cd apigee-simulator/backend
npm install
npx prisma generate
npx prisma db push
npx prisma db seed
npm run dev
# Backend listens on http://localhost:4000
```

Frontend (Terminal 2)

```
cd apigee-simulator/frontend
npm install
npm run dev
# Open http://localhost:5173 in your browser
```

Login

Field	Value
URL	http://localhost:5173
Email	admin@apigee-sim.local
Password	Apigee123!

NOTE: This is a seeded local account created by the Prisma seed script for evaluation purposes only. It is not a real credential and only exists in your own local SQLite database (backend/prisma/dev.db).

The seed script also creates the demo data referenced throughout this document: three environments (dev, test, prod), four target servers backed by real public APIs, eight demo API proxies, three developers with registered apps, two API products, two Key Value Maps, and 30 days of mock analytics. You do not need to create any of this yourself — it is already there the first time you log in.

3. The UI, Screen by Screen

The left navigation groups screens the same way real Apigee's console does: Develop (Proxies, Shared Flows), Publish (Products, Developers, Apps, Environments, Target Servers), Admin (KVMs, References, Environment Groups, Flow Hooks), and Runtime (Deployments, Trace, Analytics, Monitoring).

3.1 Login

A simple two-tab screen: **Sign in** for the seeded admin account, and **Create account** to register a new local user. There is no real OAuth here — it is a local JWT issued by the backend and stored in the browser for the session.

3.2 Dashboard

Landing page after login. Gives an at-a-glance overview of your simulated organization: proxy count, deployment status, recent trace activity, and quick links into the other modules. Use it as a jumping-off point, not a place to configure anything.

3.3 API Proxies — List

Reached via the left nav's "API Proxies" item. Shows every proxy as a table with its base path, description, and deployment status. Click **+ Create Proxy** to open a dialog asking for a **Name** and **Base Path** (e.g. name `InventoryService-v1`, base path `/inventory`) — this creates Revision 1 with an empty default ProxyEndpoint/TargetEndpoint pair, ready to configure. Click any row to open that proxy's detail page.

3.4 API Proxies — Detail

The detail page is where almost all proxy configuration happens. It has three tabs:

Overview tab

- Editable **Description** field (name and base path are fixed after creation).
- "Live endpoint by deployed environment" — for every environment this proxy is deployed to, shows the resolved, clickable URL of its **default** target endpoint. If the proxy has more than one Route Rule, a note reminds you other HTTP methods may resolve to a different target — check the Develop tab for that.

- A **Revisions** panel listing every revision with its creation time, plus a **+ Create new revision** button. Real Apigee is revision-based: editing policies/flows on an already-deployed revision does not retroactively change traffic already routed to it — create a new revision, edit that, then deploy it.

Develop tab — this is the important one

Reading top to bottom, left column:

- **Flow visualization** — a diagram of the PreFlow → Target → PostFlow chain for the *default* route. Click any policy node to open its configuration dialog.
- **Target Endpoints panel** — one card per target endpoint on this revision. Each card lets you pick a **Target Server** (from the Target Servers module) and an optional **Failover Target Server**, and shows the resolved host per environment as clickable links. **+ Add Target Endpoint** prompts for a name and appends a new, empty target endpoint — you then wire a Route Rule to it (see below). The delete button is disabled while a Route Rule still references that target, and while it is the last remaining target endpoint.
- **Route Rules panel** — a table with reorder arrows, Rule Name, Verb (blank = any), Base Path Suffix, and Target Endpoint columns, evaluated top to bottom. **+ Add Route Rule** appends a new rule defaulting to the first target endpoint. Leave the **last** rule's Verb/Path blank so it acts as the default/fallback — exactly like real Apigee's RouteRule evaluation order.
- **Conditional Flows panel** — cards with an editable Name, Verb, and Base Path Suffix, plus read-only lists of which policies are attached to that flow's request/response. **+ Add Conditional Flow** appends an empty one; attach policies to it from the Policies panel on the right (see next bullet).

Right column:

- **Policies panel** — every policy attached anywhere on this revision, showing its type and where it's attached (ProxyEndpoint PreFlow/PostFlow, or a named Conditional Flow's request/response side). **+ Add** opens the policy catalog picker; before picking, use the **"Attach new policy to"** dropdown above the list to choose the destination — ProxyEndpoint PreFlow, ProxyEndpoint PostFlow, or any Conditional Flow's request/response. Clicking an existing policy card reopens its configuration dialog to edit it; the trash icon removes it (and cleans it out of whichever flow referenced it).

WATCH OUT: The XML shown in a policy's configuration dialog is generated from the Configuration tab's fields and is for reference/export only — hand-editing that XML text does not change runtime behavior. Change the Configuration tab's fields instead; that is what the trace engine actually reads.

Deploy tab

One card per environment (dev/test/prod). If the current revision isn't deployed there, a **Deploy** button deploys it; if it is, you see its status and an **Undeploy** button. A deployment history table below shows every deploy/undeploy event with timestamps. A proxy must be deployed to an environment before the Trace tool can resolve a live target URL for it in that environment (Trace itself works even without a deployment, since it invokes the policy/flow logic directly against the revision).

3.5 Import OpenAPI

Available from the proxy detail page's header button. Paste an OpenAPI 3.0 JSON document; the importer generates one Conditional Flow per path + HTTP verb combination found in the spec. Useful as a starting point

for a proxy with many conditional routes, though you'll still want to attach policies and wire Route Rules by hand afterward.

3.6 Shared Flows & Flow Hooks

Shared Flows are reusable policy sequences (see the seeded Common-Security-Flow, which runs a MessageLogging then an AccessControl policy). A proxy invokes one explicitly with a **FlowCallout** policy. **Flow Hooks** instead attach a shared flow *globally*, per environment, at one of four points: PreProxyFlowHook, PostProxyFlowHook, PreTargetFlowHook, or PostTargetFlowHook — every proxy deployed to that environment runs it automatically. The seed data attaches Common-Security-Flow as a PreProxyFlowHook on the **prod** environment only, so any prod trace will show two extra timeline steps from that shared flow before the proxy's own PreFlow runs.

3.7 Environments & Environment Groups

Environments (dev/test/prod) are isolated deployment targets — the same proxy revision can be deployed independently to each. Each environment card shows its virtual hosts and lets you add more. **Environment Groups** (a bonus feature mirroring Apigee X) map inbound hostnames to a set of environments — the seeded default-group routes api.apigee-sim.local and *.apigee-sim.local to the test and prod environments.

3.8 Target Servers

Named backend endpoints that Target Endpoints point to by name (rather than a hardcoded URL), so the same proxy resolves to a different real backend per environment. The seed data creates four, each duplicated across dev/test/prod, all backed by real public APIs so target calls in Trace genuinely succeed:

Field	Value
CustomerService	https://jsonplaceholder.typicode.com/users
OrderService	https://jsonplaceholder.typicode.com/posts
ProductService	https://fakestoreapi.com/products
MockWeather	https://api.open-meteo.com/v1/forecast?latitude=37.77&longitude=-122.42¤t_weather=true

A fifth, deliberately unreachable server, LegacyBillingService (test environment only), exists purely to demonstrate target failover — see Section 6.4.

3.9 API Products, Developers & Apps

API Products bundle one or more proxies with a quota, scopes, and environment restrictions — the unit developers actually subscribe to (seeded: **Free-Tier**, auto-approved, 1000/day; **Premium-Tier**, manual approval, 100,000/month, monetized). **Developers** are the people who register apps; **Apps** are what actually hold a Consumer Key/Secret pair and a list of subscribed products. Open any app row to reveal its key/secret with a copy button, and use **Regenerate keys** if you need fresh credentials. These are the real key/secret pairs used in the VerifyAPIKey and OAuthV2 policy tests in Section 7.

3.10 Key Value Maps (KVM)

Simple key/value config storage, scoped to either the whole organization or a single environment (optionally marked **encrypted**, which only affects display masking here, not real encryption). The seed data includes an org-wide global-config map and a dev-secrets map scoped to the dev environment.

3.11 References

Indirect pointers to a KeyStore, TrustStore, or JWKS resource, referenced by name from security policies — most notably VerifyJWT's JWKS ref field, which points at the seeded jwks-reference entry.

3.12 Deployments

A cross-proxy view of every deployment: which revision of which proxy is live in which environment, with quick Deploy/Undeploy controls. Functionally a global version of the Deploy tab on each proxy's detail page — useful when you want to see everything deployed to, say, prod, without opening each proxy.

3.13 Analytics & Monitoring

Analytics charts request volume, status code mix, latency, and top APIs, computed from 30 days of mock analytics events the seed script generates (this is historical/synthetic data, not live trace traffic). **Monitoring** shows availability/latency/error-rate/traffic over the last hour and lets you manually simulate an alert to see how the alerting UI looks and resolves.

3.14 Settings

Local account info only — not a configuration surface for the simulator itself.

3.15 Trace — the tool you'll use most

Trace sends a simulated request through a deployed (or even just-configured) proxy and shows every policy execution step by step, with the resolved request/response and every flow variable that got set along the way. This is the primary tool used throughout Section 7's policy tests. The Request panel now mirrors Postman's request builder:

- **Proxy / Environment** — pick which deployed proxy and which environment to trace against.
- **Method** and **Path** — the path defaults to the proxy's base path; edit it to add a sub-path (e.g. /items/42) or an inline query string.
- **Query Params** — a dedicated key/value table (the Postman "Params" tab equivalent). Rows here are merged with any ?key=value typed directly into Path; if the same key appears in both, the table wins.
- An info box (once a proxy is selected) shows the resolved base path, which **Target Endpoint** this method/path combination routes to, the underlying **Target Server**, and — when the proxy has more than one Route Rule — which rule matched, with a clickable link to the actual resolved backend URL.
- **Headers** — a key/value table, same style as Query Params. The default row is pre-filled with an empty Authorization header for convenience.
- **Body** — a **Body Type** dropdown selects between none, raw (JSON), x-www-form-urlencoded, and form-data. JSON uses the familiar Monaco code editor; the two form types get their own key/value table (mirroring Postman's Body tab) and automatically set a matching Content-Type response header unless you've already set one yourself.

- **Simulate Fault** — pick a policy name attached to this proxy to force it to fail, useful for practicing fault-handling behavior without needing a genuinely invalid request.
- **Run Trace** — executes the request. Results appear on the right: a **Flow Execution Timeline** (every PreFlow/Conditional/Target/PostFlow/FlowHook step, with success/error/skipped status and duration), the resolved **Request** and **Response** (headers + body), and a **Variables Created** table listing every flow variable any policy set.
- **Recent Trace Sessions** — the last 10 traces you've run, clickable to reload that result without re-running it.

NOTE: HTTP header names are case-insensitive on the wire, and this simulator normalizes every incoming header key to lowercase before any policy runs. Type Authorization, authorization, or AUTHORIZATION in the Headers table — all three are treated identically, exactly like a real browser or Postman would send them.

4. Core Concepts: ProxyEndpoint, TargetEndpoint, Route Rules & Conditional Flows

These four pieces are the heart of how a real Apigee proxy — and this simulator — decides what runs and where a request ends up. Understanding them is the difference between a proxy that only ever does one thing, and one that behaves like a real production API gateway.

4.1 ProxyEndpoint

The client-facing side of a proxy. It defines the **PreFlow** (policies that run before routing — authentication, rate limiting, header manipulation) and **PostFlow** (policies that run after a response comes back — logging, CORS headers, response shaping), plus any number of named **Conditional Flows** and a list of **Route Rules** that decide which Target Endpoint handles the request. In this simulator's UI, this is everything in the Develop tab except the Target Endpoints panel.

4.2 TargetEndpoint

The backend-facing side. Each TargetEndpoint has its own PreFlow/PostFlow (for target-specific transformation) and points at either a named **Target Server** (recommended — lets the same proxy resolve to a different real backend per environment) or a hardcoded URL, plus an optional **Failover Target Server**. A single proxy revision can define **multiple** TargetEndpoints; which one handles a given request is decided by Route Rules.

4.3 Route Rules

A ProxyEndpoint's ordered list of rules, each pairing an optional **condition** (HTTP verb and/or a base-path suffix) with a **Target Endpoint name**. Rules are evaluated top to bottom; the first one whose condition matches (or has no condition at all) wins. A rule with no condition always matches, so convention — in real Apigee and in this simulator — is to put it **last**, as the default/fallback route. This is exactly how a single proxy can send GET requests to a read-only backend and POST/PUT/PATCH/DELETE to a different, writable one.

```
"routeRules": [
  { "name": "reads", "targetEndpoint": "read-target", "condition": { "verb": "GET" } },
  { "name": "writes", "targetEndpoint": "write-target" } // no condition -> fallback, matches everything
]
```

This is the actual Route Rule array from the seeded **MultiRouteDemo-v1** proxy (base path /items) — you'll test it directly in Section 6.2.

4.4 Conditional Flows

A named block of policies (its own request/response lists) that runs **instead of** the ProxyEndpoint's plain PreFlow/PostFlow when its condition (verb and/or base-path suffix) matches the incoming request. Use this when different HTTP methods or sub-paths on the same proxy need genuinely different policy logic — not just a different backend (that's what Route Rules are for), but different security, validation, or transformation steps entirely. For example, a conditional flow scoped to DELETE requests might attach a stricter AccessControl

policy that a plain GET never needs.

NOTE: Route Rules answer "which backend?" — Conditional Flows answer "which policies?". A single proxy commonly uses both together: a conditional flow adds an extra validation policy for POST requests, while a route rule sends those same POST requests to a different target endpoint than GET requests use.

5. Hands-On: Build an API Proxy With Multiple Target Endpoints and a Conditional Flow

This walks through building a new proxy, **InventoryService-v1**, entirely from the UI. It ends up structurally identical to the seeded MultiRouteDemo-v1 (Section 4.3), but building your own is the best way to confirm you understand every field. Budget about 10 minutes.

Step 1 — Create the proxy

- 1 Go to **API Proxies** in the left nav, click **+ Create Proxy**.
- 2 Name: **InventoryService-v1**. Base Path: **/inventory**. Click **Create**.
- 3 You land on the new proxy's detail page, Overview tab, Revision 1 — currently one empty default Target Endpoint named default and one default Route Rule pointing at it.

Step 2 — Add a second Target Endpoint

- 1 Switch to the **Develop** tab.
- 2 In the **Target Endpoints** panel, click **Add Target Endpoint**. When prompted for a name, enter **read-target**.
- 3 On the new **read-target** card, set **Target Server** to **ProductService**.
- 4 On the existing default card, rename its role conceptually to your write path: set its **Target Server** to **CustomerService**. (You can leave its name as default — Route Rules reference it by whatever name it already has.)

Step 3 — Add a Route Rule so GET goes to read-target

- 1 In the **Route Rules** panel, click **Add Route Rule**. A new rule appears above the existing default rule.
- 2 Use the up-arrow to move the new rule **above** the existing default rule — rules are evaluated top to bottom, and the unconditional one must stay last.
- 3 On the new (now first) rule: set **Verb** to **GET**, leave **Base Path Suffix** blank, and set **Target Endpoint** to **read-target**.
- 4 Leave the remaining rule's Verb blank and its Target Endpoint as default — this is now your fallback for POST/PUT/PATCH/DELETE.

NOTE: The Route Rules panel blocks deleting a Target Endpoint that's still referenced by a rule, and blocks removing the very last remaining rule or target endpoint — you cannot accidentally leave the proxy with nowhere to route.

Step 4 — Add a Conditional Flow for a single-item lookup

- 1 In the **Conditional Flows** panel, click **Add Conditional Flow**.
- 2 Rename it to **get-single-item**. Set **Verb** to **GET** and **Base Path Suffix** to **/inventory/** (matches any path containing that suffix, e.g. **/inventory/7**).
- 3 In the **Policies** panel, set the **"Attach new policy to"** dropdown to **Conditional Flow "get-single-item" (request)**.

- 4 Click **+** **Add**, choose **ExtractVariables** from the catalog, leave its defaults (Source: request, Variable Name: extracted.value, JSONPath: \$.id), and save.

Your proxy now has: two target endpoints (read-target -> ProductService, default -> CustomerService), a route rule sending GET to read-target and everything else to default, and a conditional flow that only engages for GET requests whose path contains /inventory/. Section 6 tests every one of these paths.

6. Testing Route Rules, Conditional Flows, Target Endpoints & Failover

All of these tests use the **Trace** tool. For each, set the Proxy and Environment first, then watch the info box under Path (Section 3.15) confirm which Target Endpoint and Route Rule matched before you even click Run Trace — that alone proves route resolution is working correctly.

6.1 Test: GET routes to read-target (MultiRouteDemo-v1)

Field	Value
Proxy	MultiRouteDemo-v1
Environment	test
Method / Path	GET /items
Headers / Body	none needed

Expected info box: **Target endpoint: read-target, Target server: ProductService**, matched rule **reads**.

PASS CRITERIA: Run Trace succeeds (200) with a JSON array of products from fakestoreapi.com in the Response panel, and the timeline's Routing step reads "Routed GET /items to target endpoint \"read-target\"".

6.2 Test: POST routes to write-target instead

Field	Value
Proxy	MultiRouteDemo-v1
Environment	test
Method / Path	POST /items
Body	Body type raw (JSON): {} (empty object is fine)

Expected info box: **Target endpoint: write-target, Target server: CustomerService**, matched rule **(fallback)** — because the fallback rule has no condition, the label shown is "(fallback)" rather than a rule name lookup failure. This single change of HTTP method, with everything else identical, is the proof that Route Rules — not the proxy as a whole — decide the backend.

6.3 Test: your own InventoryService-v1 (built in Section 5)

Repeat 6.1/6.2 against the proxy you built:

- **GET /inventory** -> should resolve to read-target / ProductService.
- **POST /inventory** -> should resolve to default / CustomerService.

- **GET /inventory/7** -> should still resolve to read-target (Route Rules only look at verb here), *and* the Flow Execution Timeline should show an extra step: Conditional Flow: get-single-item (Request) running **ExtractVariables**, plus extracted.value appearing in the Variables Created table — proving the conditional flow's base-path-suffix condition matched.
- **GET /inventory** (no suffix) -> the same ExtractVariables step should be **absent** from the timeline, since the conditional flow's path condition didn't match.

WATCH OUT: If step 3 shows the conditional flow running but step 4 also shows it, double check the Base Path Suffix field — it should be /inventory/ (with the trailing slash) so /inventory alone doesn't contain that substring.

6.4 Test: Target Server failover (FailoverDemo-v1)

FailoverDemo-v1 (base path /failover-demo, deployed to test) points its target endpoint at LegacyBillingService — a target server whose host is a deliberately unresolvable domain — with ProductService configured as its Failover Target Server.

Field	Value
Proxy	FailoverDemo-v1
Environment	test
Method / Path	GET /failover-demo

Expected timeline: a **Target Request** step with status **error** for the primary call to LegacyBillingService, immediately followed by a second **Target Request (Failover)** step with status **success** calling ProductService instead, and the final response is a normal 200 with product data — the fault never reaches the client because failover absorbed it.

NOTE: This proves failover, not just that ProductService works: if you temporarily clear the Failover Target Server field on this proxy's target endpoint and re-run the same trace, you should instead see the request end in a 502 with faultstring "Target and failover target both unreachable" — then restore the field afterward.

6.5 Quick reference: what each Trace info-box line proves

Field	Value
Target endpoint: X	Route Rules resolved correctly for this method/path
Target server: Y	The target endpoint is wired to a real Target Server record
Matched route rule: Z	Which specific rule in the ordered list fired (only shown when >1 rule exists)
Resolved URL (link)	The exact backend host this environment will actually call

7. Policy-by-Policy Testing Guide

Every policy type in the catalog, grouped by category. For each: what it does, the fields you'll see in its Configuration tab, how to attach and test it, a sample Trace request, and the exact response/behavior you should see if it's working — taken directly from this simulator's executor logic, not guessed.

NOTE: "Attach" always means: open the proxy's Develop tab, set "Attach new policy to" to the flow you want, click + **Add** in the Policies panel, choose the policy type, fill in the Configuration tab, and Save.

7.1 Security Policies

VerifyAPIKey

Validates an API key against a registered Developer App. Already attached to **CustomerService-v1** (query param apikey) and **ProductCatalog-v1** (header x-api-key).

Field	Value
apiKeyLocation	query or header
apiKeyParam	parameter/header name, default apikey

Get a real key: Apps page -> open **Acme-Mobile-App** -> copy its Consumer Key.

Field	Value
Proxy / Env	CustomerService-v1 / test
Method / Path	GET /customers
Query Params	apikey = <paste the copied Consumer Key>

PASS CRITERIA: Timeline shows VerifyAPIKey as success with message "API key verified for app \"Acme-Mobile-App\"", and Variables Created includes verifyapikey.developer.app.name and verifyapikey.apiprduct.names. Omit the query param entirely and re-run: expect a 401 with faultstring "Failed to resolve API Key variable request.apikey".

OAuthV2 — GenerateAccessToken

Issues a real, server-side OAuth2 token for client_credentials grant. Already attached to **OAuthToken-v1** (base path /oauth/token).

Field	Value
Proxy / Env	OAuthToken-v1 / test
Method / Path	POST /oauth/token

Body (raw JSON)	{ "client_id": "<Acme-Mobile-App Consumer Key>", "client_secret": "<its Consumer Secret>" }
-----------------	---

PASS CRITERIA: Response body is {"access_token": "...", "token_type": "Bearer", "expires_in": 3600, "scope": "read"} with a real, freshly-minted token; the timeline shows the Target Request step as **skipped** ("response was produced directly by a policy") since OAuthV2 answers the request itself. Copy the access_token for the next test.

OAuthV2 — VerifyAccessToken

Validates a previously issued token. Already attached to **OAuthProtected-v1** (base path /secure), configured to read from the Authorization header by default — but the Location can be switched to Query Parameter in its Configuration tab.

Field	Value
Proxy / Env	OAuthProtected-v1 / test
Method / Path	GET /secure
Headers	Authorization = Bearer <token from the previous test>

PASS CRITERIA: Success message: "Access token verified (client_id ...)". Change accessTokenLocation to Query Parameter in the policy's Configuration tab, re-attach the token as ?access_token=... instead, and confirm it still passes -- this is the exact gap the token-location feature was added for.

WATCH OUT: Send no Authorization header at all: expect 401, faultstring "Invalid Access Token", message "Missing access token - expected it in the header \"Authorization\"". Send an expired/garbage token: expect 401 "Access Token expired" or "Invalid Access Token" depending on which check fails.

VerifyJWT

Not attached to any seeded proxy by default — attach it to a PreFlow yourself to test it (e.g. on your InventoryService-v1 from Section 5, or any other proxy).

Field	Value
source	flow variable holding the JWT, default request.header.Authorization
algorithm	RS256 / HS256 / ES256
issuer / audience	expected iss / aud claims
jwtRef	name of a References entry (seeded: jwt-reference)

Because this simulator checks structure/claims rather than a real signature, you can build a throwaway test JWT quickly. A JWT with header {"alg":"RS256"} and payload

`{"iss":"https://accounts.example.com","aud":"my-api"}` (base64url-encode each part and join with dots, third segment can be any string) will pass with default policy settings.

WATCH OUT: Send a value with fewer than 3 dot-separated segments (e.g. just "abc"): expect 401, faultstring "JWT is not valid", errorcode steps.jwt.InvalidToken. This is the fastest way to prove the policy is wired up correctly before building a real token.

BasicAuthentication

Decodes (or encodes) an HTTP Basic Authorization header. Default config decodes into `request.formparam.username/password`.

Field	Value
operation	Decode or Encode
user / password	flow-variable refs, default <code>request.formparam.username / .password</code>

Attach with `operation=Decode` to a PreFlow, then test with a real Basic header:

Field	Value
Headers	Authorization = Basic YWxpY2U6c2VjcmV0MTIz (base64 of <code>alice:secret123</code>)

PASS CRITERIA: Success message "Basic auth decoded"; Variables Created shows `request.formparam.username = alice` and `request.formparam.password = ****` (masked). Omit the header entirely: expect 401 "Missing Basic Authorization header".

Switch operation to `Encode` to test the other direction: set Body Type to `x-www-form-urlencoded` with fields `username=alice`, `password=secret123` (matching the policy's default refs) and run with no Authorization header set. The policy resolves those form fields and writes a real Basic header onto the outgoing request itself.

CORS

Adds Access-Control-* response headers. Already attached to **WeatherProxy-v1** (PreFlow, but it writes response headers regardless of phase).

Field	Value
Proxy / Env	WeatherProxy-v1 / test
Method / Path	GET /weather

PASS CRITERIA: Response Headers panel includes `Access-Control-Allow-Origin: *`, `Access-Control-Allow-Methods: GET,PUT,POST,DELETE,OPTIONS`, and `Access-Control-Allow-Headers` with the configured list.

AccessControl

Allow/deny by client IP or CIDR. Used inside the seeded Common-Security-Flow shared flow (action DENY, ipList 198.51.100.0/24) — attached as a PreProxyFlowHook on the **prod** environment only.

Field	Value
Proxy / Env	any proxy / prod

NOTE: Trace against prod always uses simulated client IP 203.0.113.1 (Trace's fixed clientIp), which is not in 198.51.100.0/24, so AccessControl passes and you'll see two extra timeline steps before the proxy's own PreFlow: SF-MessageLogging then SF-AccessControl, both success.

7.2 Traffic Management Policies

SpikeArrest

Smooths bursts to a steady rate. Already attached to **CustomerService-v1** (50ps), **OrderService-v1** (100pm), **WeatherProxy-v1** (20ps), and **FailoverDemo-v1** (50ps).

Field	Value
rate	e.g. 100ps (per second) or 30pm (per minute)
identifierRef	optional, per-client limiting

Test by firing more requests than the configured rate allows in a short window (Trace has no built-in load generator, so run the same trace repeatedly in quick succession, or lower the rate to something easy to trip, e.g. 1pm, save, then run twice within the same minute).

PASS CRITERIA: A tripped SpikeArrest returns 429, faultstring "Spike arrest violation", errorcode policies.ratelimit.SpikeArrestViolation, and every policy after it in the flow is skipped.

Quota

Enforces a consumption ceiling over an interval. Already attached to **OrderService-v1** (1000/hour) and **ProductCatalog-v1** (5000/day).

Field	Value
allowCount	max requests
interval / timeUnit	e.g. 1 / hour

Lower allowCount to something small (e.g. 3) on a test proxy, then run the same trace 4 times in a row.

PASS CRITERIA: The 4th run returns 429, faultstring "Quota Violation". Successful runs set ratelimit.Quota.count / ratelimit.Quota.allowed in Variables Created, so you can watch the count climb 1, 2, 3 before it trips.

ConcurrentRateLimit

Caps in-flight concurrency (independent of SpikeArrest/Quota's throughput limits). In this simulator it always reports as within limit, since Trace runs one request at a time -- attach it and confirm the message reads "Concurrency simulated within limit of <maxConcurrency>".

Field	Value
maxConcurrency	e.g. 10

ResponseCache

Caches full responses for a TTL. Already attached to **ProductCatalog-v1** (120s TTL).

Field	Value
Proxy / Env	ProductCatalog-v1 / test
Method / Path	GET /products

PASS CRITERIA: First run: message "Cache miss - will populate cache after target call", responsecache.cachehit = false, and a normal Target Request step runs. Run the exact same trace again within 120 seconds: message "Response served from cache (target call will be skipped)", responsecache.cachehit = true, and the timeline's Target Request step now shows status **skipped** with message "Target call skipped - response served from cache".

PopulateCache & LookupCache

The manual read/write pair underlying ResponseCache. Attach PopulateCache to a PostFlow to write, and LookupCache to a PreFlow (on a later trace) to read the same key back.

Field	Value
cacheResource / cacheKey	shared name between the two policies
assignTo (LookupCache)	flow variable to receive the cached value

PASS CRITERIA: After a PopulateCache run, a subsequent LookupCache run against the same path sets your assignTo variable to the previously cached response body and reports message "Cache hit"; against a path never populated, it reports "Cache miss" and sets the variable to null.

7.3 Mediation Policies

AssignMessage

Sets headers and/or a payload on the request or response. Already attached to **CustomerService-v1** (PostFlow, adds response header X-Powered-By) and **OrderService-v1** (PreFlow, adds request header X-Client).

Field	Value
assignTo	request or response

setHeaders	one "Name: value" pair per line
setPayload	optional JSON body to set
Field	Value
Proxy / Env	CustomerService-v1 / test
Method / Path	GET /customers

PASS CRITERIA: Response Headers panel includes x-powered-by: Apigee-Simulator (header names are normalized to lowercase, matching how the request-side lookups work).

ExtractVariables

Pulls a JSONPath value out of the request or response body into a flow variable. Used in the Section 5 build exercise's get-single-item conditional flow.

Field	Value
source	request or response
variableName	e.g. extracted.value
jsonPath	e.g. \$.id
Field	Value
Proxy / Env	InventoryService-v1 (Section 5) / test
Method / Path	GET /inventory/7

NOTE: Variables Created includes extracted.value set to whatever \$.id resolves to in the response body (fakestoreapi.com product listings are an array, so with the default JSONPath against an array response the value may be undefined -- change jsonPath to \$.0.id to pull the first product's id specifically, and re-run to see a real numeric value).

ServiceCallout

Makes a side HTTP call mid-flow without touching the main request/response. Not attached to any seeded proxy -- attach it to any PreFlow to test; the default target URL is a real, reachable public API.

Field	Value
targetUrl	default https://jsonplaceholder.typicode.com/todos/1
method	GET/POST/PUT/DELETE
responseVariable	e.g. calloutResponse

PASS CRITERIA: Variables Created includes calloutResponse populated with the real JSON from that URL (id, title, completed fields), and the message reads "ServiceCallout to ... returned 200" -- proving it's a genuine outbound call, not a stub.

FlowCallout

Invokes a Shared Flow by name. Set sharedFlowName to the seeded Common-Security-Flow and attach to any PreFlow to test.

NOTE: Message reads "Shared flow \"Common-Security-Flow\" invoked (see nested steps)" -- in this simulator FlowCallout reports success without inlining the shared flow's own steps into the timeline (that inlining only happens automatically for Flow Hooks); treat it as confirmation the reference resolved correctly.

JavaScript

Runs a snippet against a sandboxed context object with getVariable/setVariable, mirroring real Apigee's JavaScript policy. Not attached to any seeded proxy.

Field	Value
source	default: context.setVariable('custom.executed', true);

PASS CRITERIA: Variables Created includes custom.executed = true after a successful run. Change the source to something that throws (e.g. undefinedFunction()) and re-run: expect 500, faultstring "JavaScript runtime error", message "JavaScript error: ...".

MessageLogging

Formats and "ships" a log line to a syslog host (simulated -- no real network call is made). Attach to a PostFlow so {response.status.code} has a real value to substitute.

Field	Value
messageTemplate	supports {organization.name}, {request.uri}, {response.status.code}

NOTE: Message reads "Logged to syslog logs.example.com:514 -> \"apigee-sim-org /your/path 200\" with the placeholders substituted -- there is no separate log file to check, the timeline message itself is the proof.

RaiseFault

Deliberately halts the flow with a custom status/body. Attach to any PreFlow to test error-handling paths on purpose.

Field	Value
-------	-------

statusCode / reasonPhrase	default 400 / Bad Request
errorMessage	default {"error": "Invalid request"}

PASS CRITERIA: Response is exactly the configured status/body, and every policy listed after RaiseFault in that flow is skipped in the timeline -- confirm this by attaching a second, harmless policy after it and checking its status shows "skipped".

7.4 Transformation Policies

JSONToXML / XMLToJSON

Convert the request or response body between JSON and XML. Attach JSONToXML (source=response) to a PostFlow on any proxy with a JSON-returning target, e.g. ProductCatalog-v1.

PASS CRITERIA: Response Headers shows content-type: application/xml and the Body panel shows an XML-serialized version of the product list wrapped in a <root> element. Swap in XMLToJSON on a proxy whose response is already XML text to see the reverse: content-type flips to application/json and the body becomes a parsed object.

XSL Transform

Applies a named XSL stylesheet. This simulator treats it as a passthrough simulation (no real XSLT engine) -- attach it and confirm the message reads "XSL stylesheet \"transform.xml\" applied to response (simulated passthrough)" to prove the policy is wired up; body content is unchanged.

7.5 Extension Policies

All four Extension connectors are simulated (no real Salesforce/BigQuery/Cloud Storage/Pub-Sub credentials are involved) -- they demonstrate the Extension policy category's Connector + Action + Object XML pattern and always succeed with a plausible fake result.

SalesforceConnector

Field	Value
operation	query/create/update/delete
subject	default Account

PASS CRITERIA: Variables Created includes connector.salesforce.result = {"id": "SF...", "success": true}.

BigQueryConnector

Field	Value
-------	-------

projectId	default my-gcp-project
query	default SELECT * FROM dataset.table LIMIT 10

PASS CRITERIA: Variables Created includes connector.bigquery.rows, a two-row sample array.

CloudStorageConnector

Field	Value
bucket / objectName	default my-bucket / file.json
operation	read/write/delete/list

PASS CRITERIA: Message reads "Cloud Storage read on gs://my-bucket/file.json simulated".

PubSubConnector

Field	Value
topic	default my-topic
operation	publish/pull

PASS CRITERIA: Variables Created includes connector.pubsub.messageId, a random simulated message ID.

8. Appendix: Seed Data Reference

Everything below already exists the first time you log in -- created by backend/prisma/seed.ts.

8.1 Environments

Field	Value
dev	Development, type BASE
test	Test, type BASE
prod	Production, type COMPREHENSIVE

8.2 Demo API Proxies

Field	Value
CustomerService-v1	/customers -- VerifyAPIKey(query) + SpikeArrest(50ps) + AssignMessage(response header)
OrderService-v1	/orders -- VerifyJWT + SpikeArrest(100pm) + Quota(1000/hr) + AssignMessage(request header)
ProductCatalog-v1	/products -- VerifyAPIKey(header x-api-key) + Quota(5000/day) + ResponseCache(120s)
WeatherProxy-v1	/weather -- CORS + SpikeArrest(20ps), public, no auth
FailoverDemo-v1	/failover-demo -- unreachable primary target + ProductService failover
OAuthToken-v1	/oauth/token -- OAuthV2 GenerateAccessToken (client_credentials)
OAuthProtected-v1	/secure -- OAuthV2 VerifyAccessToken
MultiRouteDemo-v1	/items -- Route Rules: GET -> read-target (ProductService), else -> write-target (CustomerService)

8.3 Target Servers (x3 per environment, plus one failover demo)

Field	Value
CustomerService	https://jsonplaceholder.typicode.com/users
OrderService	https://jsonplaceholder.typicode.com/posts
ProductService	https://fakestoreapi.com/products
MockWeather	https://api.open-meteo.com/v1/forecast?...
LegacyBillingService (test only)	deliberately unreachable, for failover testing

8.4 API Products, Developers & Apps

Field	Value
Free-Tier	auto-approved, 1000/day, proxies: CustomerService-v1, WeatherProxy-v1
Premium-Tier	manual approval, 100,000/month, all four core proxies, monetized
Developers	grace.hopper@example.com, ada.lovelace@example.com, alan.turing@example.com
Apps	Acme-Mobile-App (Free-Tier), Analytical-Dashboard (Premium-Tier), Bletchley-Integration (both)

Consumer Key/Secret values are randomly generated at seed time -- always copy the live value from the Apps page rather than reusing one from an earlier run.

8.5 Key Value Maps & References

Field	Value
global-config (org-wide)	support.email, brand.name
dev-secrets (dev environment)	api.upstream.secret
jwt-reference	used by VerifyJWT's JWKS ref field

9. Troubleshooting Quick Reference

Getting a 401 "bearer token missing" or "invalid access token" when the header looks right

Header names are case-insensitive everywhere in this simulator -- Authorization, authorization, and AUTHORIZATION are all treated the same. If a 401 persists, check: is the token actually prefixed with "Bearer " (with the space)? Is accessTokenLocation on the OAuthV2 policy set to the location you're actually sending the token in (Header vs Query Parameter)? Did the token come from a GenerateAccessToken call against the same simulator instance (tokens are server-side and not portable across a fresh database)?

Edited a policy's XML directly and the change didn't take effect

This is expected -- the XML shown is generated from the Configuration tab and is for reference/export only. Make the change on the Configuration tab's fields instead; that is what the trace engine reads at runtime.

A Route Rule or Conditional Flow doesn't seem to match

Verb comparison is case-insensitive but exact (GET must match GET, not just any non-empty value). Base Path Suffix uses a plain substring check against the full request path, not a template match -- "/items/{id}" as literal text will never match a real path like "/items/42"; use a plain suffix like "/items/" instead.

Target call fails with a network error in Trace

The target servers point at real public APIs (jsonplaceholder.typicode.com, fakestoreapi.com, open-meteo.com) -- this requires outbound internet access from wherever the backend process runs. If your machine is offline or behind a restrictive proxy, target calls will fail even though the policy logic is correct.